Beyond Fault Localization: A Trajectory-Level Study of LLM Agents for Microservice Root Cause Analysis

QISHENG LU, The Chinese University of Hong Kong, Shenzhen, China
AOYANG FANG*, The Chinese University of Hong Kong, Shenzhen, China
JUNJIELONG XU, The Chinese University of Hong Kong, Shenzhen, China
JIN'AO SHANG, Xi'an Jiaotong University, China
SONGHAN ZHANG, The Chinese University of Hong Kong, Shenzhen, China
YIFAN YANG, The Chinese University of Hong Kong, Shenzhen, China
XIAOCHUAN YAN, The Chinese University of Hong Kong, Shenzhen, China
PINJIA HE*, The Chinese University of Hong Kong, Shenzhen, China

arXiv:2608.21310v2 [cs.SE] 9 Oct 2026

Automated microservice root cause analysis (RCA) is usually evaluated by determining whether the final answer localizes the fault source. However, this kind of evaluation is insufficient for on-call site reliability engineers (SREs), who need the evidence and reasoning behind the fault. Evaluating the diagnostic process is challenging because agent trajectories interleave free-form natural-language reasoning with heterogeneous tool calls and telemetry results, making it difficult to assess agents' behavior.

To assess how well RCA agents ground their diagnoses in evidence, we propose a trajectory-analysis methodology that characterizes evidence navigation and utilization, SRE-oriented diagnostic intent, and recurring failure modes. Our key insight is that fault-propagation paths provide a causal reference for assessing where agents investigate and how they use evidence. Using this methodology, we analyze 5,000 trajectories from ten framework–backbone configurations on RCABench, covering two backbone models and nine agent frameworks. We find that correct localization can hide deficiencies in causal reconstruction. Successful investigations use retrieved evidence to guide subsequent queries, consider a broader range of diagnostic intents, and investigate the problem more deeply. Failed investigations drift toward unrelated services or repeat shallow queries, with recurring failure patterns arising from omitted evidence, misinterpreted evidence, and unsupported reasoning.

To test whether these mined failure patterns can guide improvement, we instantiate a lightweight defense prototype targeting the three failure families. Across two held-out benchmarks, two backbone models, and six diagnostic frameworks, the cataloged failure patterns recur; the prototype reduces their prevalence and raises Acc@1 by 3.5–35.1 percentage points across all tested configurations, suggesting that the mined failures both generalize and are actionable. Taken together, our findings motivate a process-level view of automated RCA, in which diagnostic trajectories reveal whether a diagnosis is not only correct, but also well-grounded and open to improvement.

Additional Key Words and Phrases: Root Cause Analysis, Microservices, LLM Agents, AIOps

1 Introduction

Microservice architectures underpin large-scale cloud applications [22], where local faults can propagate across loosely coupled services and trigger cascading failures. When such incidents occur, on-call site reliability engineers (SREs) must diagnose the fault source from distributed traces,

*Pinjia He and Aoyang Fang are co-corresponding authors.

Authors' Contact Information: Qisheng Lu, The Chinese University of Hong Kong, Shenzhen, Shenzhen, China, qishenglu@link.cuhk.edu.cn; Aoyang Fang, The Chinese University of Hong Kong, Shenzhen, Shenzhen, China, aoyangfang@link.cuhk.edu.cn; Junjielong Xu, The Chinese University of Hong Kong, Shenzhen, Shenzhen, China, siyuexi@foxmail.com; Jin'ao Shang, Xi'an Jiaotong University, Xi'an, China, jinao_s@stu.xjtu.edu.cn; Songhan Zhang, The Chinese University of Hong Kong, Shenzhen, Shenzhen, China, songhanzhang@link.cuhk.edu.cn; Yifan Yang, The Chinese University of Hong Kong, Shenzhen, Shenzhen, China, yifanyang6@link.cuhk.edu.cn; Xiaochuan Yan, The Chinese University of Hong Kong, Shenzhen, Shenzhen, China, xiaochuanyan@link.cuhk.edu.cn; Pinjia He, The Chinese University of Hong Kong, Shenzhen, Shenzhen, China, hepinjia@cuhk.edu.cn.

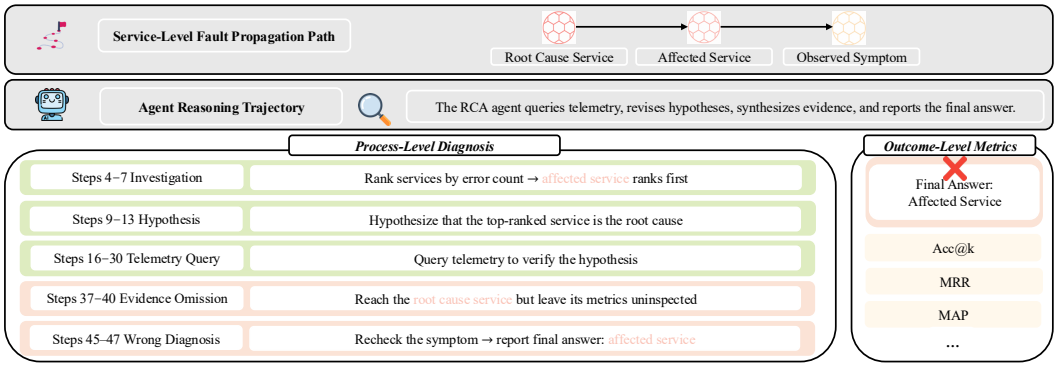


Fig. 1. Outcome-level metrics (right) record only the final answer and its correctness score. The diagnostic trajectory (left) exposes the process behind the same misdiagnosis: the agent navigates to the root cause service (Steps 37–40) but leaves its metrics uninspected, then rechecks the symptom and reports the affected service as its answer. The annotated fault propagation path (top) provides the process-level ground truth against which such failures can be identified.

logs, and metrics before remediation can begin [20]. Because remediation acts on a diagnosis, the SRE needs not only a root-cause label but also the evidence and reasoning that make it trustworthy.

Automated root cause analysis (RCA) has produced a large body of methods [12, 15, 66]. Benchmark-driven studies usually turn RCA into a service-localization task: predict which service caused the incident. Top-k localization accuracy (Acc@k), mean reciprocal rank, and mean average precision make this task comparable across methods, but they collapse the diagnostic process into a final label [13, 33]. That collapse matters because the same correct label can be supported by strong or weak evidence, and a wrong label is only useful for continuous improvement if its failed reasoning can be inspected (Fig. 1). When such reasoning failures recur across incidents, they form reusable failure modes that can feed back into the diagnostic process.

Tool-using LLM systems make this missing diagnostic process newly visible. General-purpose coding and deep-research agents now plan over many steps, invoke tools, and synthesize evidence [3, 10]. LLM-based RCA methods follow the same pattern by querying telemetry and revising hypotheses during diagnosis [7, 18, 32, 36, 37, 45, 51, 57, 61, 68]. Because these agents act through observable steps, their runs leave trajectories: records of agent messages, telemetry queries, and tool-returned evidence. Such trajectories can expose how an agent investigates an incident, not only what answer it returns.

However, raw trajectories cannot be evaluated directly as process-level evidence. Prior RCA work studies causal graphs and fault propagation for localization [12, 15, 66], and agent studies analyze tool traces [3, 10], but causal graphs capture statistically inferred service dependencies, not the incident-specific route a fault actually traveled, and agent-trace analyses lack a domain-level ground truth to evaluate against. Evaluating diagnostic trajectories in RCA therefore requires a reference for what the diagnostic process should have recovered. For our setting, that reference is a manually validated service-level fault propagation path: the route by which a fault moves from the faulty service to the observed symptom through service-dependency hops. RCABench [14], the microservice benchmark used for our main study, provides final root-cause labels but not these process-level annotations. Even with such paths, trajectories still need normalization because heterogeneous agents expose different trace formats and tool interfaces. Without both process-level

ground truth and normalized steps, existing metrics cannot say whether an agent navigated relevant evidence, what operational purpose each step served, or why wrong diagnoses recurred.

We address this gap with a trajectory-level analysis methodology built around service-level fault propagation paths. These paths define the fault-impact surface: the services and edges through which a fault reaches the observed symptoms. To compare agent behavior against this surface, we normalize each run into agent messages, telemetry queries, and tool-returned evidence. The methodology applies outcome-level metrics to all nine frameworks and adds step-level trajectory analysis where the shared telemetry interface makes tool traces comparable. The resulting analysis first checks whether a run finds the root service and reconstructs the propagation path. The analysis then explains the route the agent took by mapping how the agent navigates the fault-impact surface, labeling SRE diagnostic intent and coding the failure modes behind wrong diagnoses. Together, these steps make RCA trajectories comparable across frameworks and suitable for mining recurring diagnostic failures.

With this methodology in place, the study pursues two questions. The first is *characterization*: what separates a sound diagnostic process from a failing one? We pursue it at three progressively deeper levels (RQ1–RQ3), moving from the outcome and cost of localization, to how an agent navigates the fault-impact surface and what diagnostic intent drives each query, to why wrong diagnoses occur. We compare nine frameworks under a shared backbone model and isolate the backbone-model effect by rerunning one framework on a second model. The trajectory-level analysis (RQ2–RQ3) focuses on the four frameworks whose tool calls are SQL queries against a shared telemetry store, because they expose comparable action traces and, as the stronger performers, yield the most informative diagnostic trajectories to analyze. The second is *intervention and validation* (RQ4–RQ5): do the characterized failure modes recur beyond the original study setting, and can they guide effective interventions? We use these failure modes to design DiagGuard, a lightweight defense prototype that augments the diagnostic agent with a grounding defense (the *Grounder*) and a verification defense (the *Verifier*). We examine failure-mode recurrence and changes in prevalence, evaluate whether DiagGuard improves localization across datasets, backbone models, and frameworks, and assess how the two defenses contribute individually and jointly.

Our study yields three core findings. 1) Localization difficulty is uneven, accuracy falls as the causal chain deepens, and reconstructing how a fault propagates between services is harder than identifying which services it reaches, even when the fault source is correctly localized. 2) Success and failure hinge on the diagnostic process itself: effective runs stay on the fault-impact surface and act on the evidence they retrieve, and the breadth of their intent repertoire tracks how deep that search reaches; failing runs drift off the surface, leave decisive evidence unused, or circle in shallow loops. 3) Wrong diagnoses collapse into a small, evidence-handling failure taxonomy; an intervention distilled from these modes raises localization accuracy across datasets, backbone models, and frameworks and reduces the modes' prevalence, while the composition of failure modes remains more similar across backbones within the same framework than across frameworks.

This paper makes the following contributions:

- **Annotated propagation-path dataset.** We annotate and manually validate service-level fault propagation paths for a microservice RCA benchmark, supplying process-level ground truth beyond final root-cause labels.
- **Trajectory-level analysis methodology.** We develop a propagation-path-grounded methodology that normalizes heterogeneous RCA trajectories against annotated propagation paths, with adapted skills for mining diagnostic failure modes across diverse LLM-based agent frameworks.
- **Process-level empirical study.** We analyze 5,000 reasoning trajectories across ten framework–backbone configurations (nine frameworks under a shared backbone model, one rerun on a second model), linking diagnostic behavior and failure modes to final RCA outcomes.

- **Failure-mode-derived intervention.** We use the mined failure modes to design DiagGuard and validate it across datasets, backbone models, and frameworks, with failure-mode analysis on 1,368 additional trajectories confirming that the catalogued modes recur and the intervention generalizes beyond the original characterization setting.

We will release all artifacts, including the annotated propagation-path dataset, trajectory-analysis pipeline, failure-mode codebook, and DiagGuard prompts.

2 Related Work

We review traditional and LLM-based RCA (Sections 2.1 and 2.2), the general-purpose agents we repurpose for RCA (Section 2.3), and trajectory-analysis techniques (Section 2.4). Across these lines, general-purpose deep-research and coding agents have not been systematically compared with RCA-specific designs on end-to-end RCA, and diagnostic processes are either not exposed or not evaluated against process-level ground truth.

2.1 Traditional RCA in Microservice Systems

Automated RCA research spans four families of methods: statistical correlation [44, 72], graph-based fault propagation [55, 58], causal inference [5, 17, 30, 40, 65], and learning-based frameworks over multimodal observability data [28, 59, 71]. These methods return a root-cause ranking rather than a reasoning process; LLM-based and agent-based methods have recently emerged as a more process-oriented direction [12].

2.2 LLM-Based RCA for Microservices

The application of large language models to microservice RCA has produced two distinct lines of work, following the *workflow*-versus-*agent* distinction [2].

Workflow approaches. These methods preprocess multimodal telemetry into a structured prompt for single-pass inference. They target classification and confidence estimation [7, 24, 60] or explanation and remediation [1, 19, 25, 49, 69], or enrich the prompt with structured knowledge such as mined code or offline causal graphs [32, 34]. A single snapshot of system state, however, cannot capture the progressive, hypothesis-driven nature of expert fault diagnosis.

Agentic approaches. These methods reason over multi-turn loops of diagnostic tool calls, telemetry queries, and hypothesis refinement, and differ in how they organize the diagnostic search. *Iterative diagnosis* agents drive a ReAct-style loop over telemetry, refining predictions through self-consistency or alternative prompting [43, 54, 57]. *Graph-structured reasoning* constrains exploration with belief graphs, evidence graphs, or service-dependency graphs [36, 51, 61–63]. *Multi-agent orchestration* coordinates multiple agents or encodes operational procedures via consensus mechanisms, behavior trees, or state machines [6, 18, 39, 48, 68, 70]. *Knowledge and experience accumulation* reuses historical diagnostic knowledge through RAG, graph knowledge bases, or self-evolving skills [26, 37, 45, 47]. *RCA-specific fine-tuning* trains the backbone on diagnostic data via SFT and RL [56, 64]; unlike the off-the-shelf agents our study examines, these require task-specific training.

These methods are nonetheless assessed almost exclusively at the outcome level, leaving the diagnostic process unexamined for lack of process-level ground truth.

2.3 Advanced Agents

General-purpose deep-research and coding agents now match or exceed domain-specific methods on knowledge-intensive tasks, raising the question of whether investigative architecture matters more than domain specialization. The DeepResearch Bench [10, 31] provides the first systematic evaluation of deep-research agents (DRAs); on the open-source side, AIQ [38], TaskWeaver [41], and ThinkDepth.ai [50] are representative frameworks, which we repurpose for RCA under the

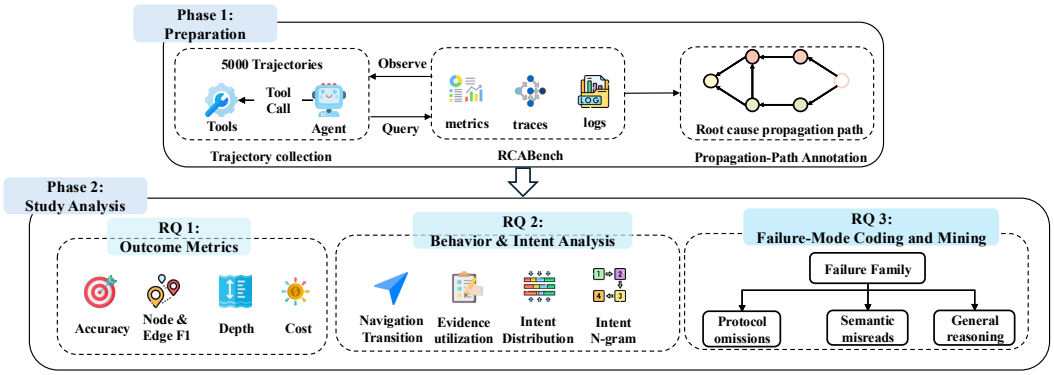


Fig. 2. **Study overview.** *Phase 1* collects 5,000 agent reasoning trajectories on RCABench: each agent queries telemetry through tool calls, and we annotate every case's fault propagation path. *Phase 2* characterizes them at three deepening levels: RQ1 outcome and cost (Section 5.1), RQ2 behavior and intent (Section 5.2), and RQ3 failure-mode families (Section 5.3); and DiagGuard (RQ4–RQ5) appears in Fig. 6a.

same harness (Section 3.1). Commercial coding agents such as ClaudeCode (Anthropic's Claude Code) integrate with development environments to navigate codebases and iterate on solutions; we adapt one to RCA through the same harness, comparing it directly with RCA-specific agents.

2.4 Agent Trajectory Analysis

Inspecting agent trajectories has attracted growing interest: tools visualize and debug agent behavior [11, 35], and a parallel line mines historical trajectories to optimize agents [9, 23, 46]. For failure understanding, Bouzenia and Pradel [3] and Zhang et al. [67] analyze trajectories of software-engineering and multi-agent systems. Closest to ours, three concurrent efforts target RCA agents: Riddell et al. [42] study post-detection RCA, where externally extracted anomalies are analyzed by agents over a static system graph, Kim et al. [27] diagnose trajectories from a single OpenRCA architecture across multiple models, and STAR [53] repairs failing diagnostic stages post-hoc. Independently, OpenRCA 2.0 [13] annotates step-level causal processes and reports that 76.0% of agents identify a correct root-cause service yet only 61.5% follow a verified propagation path, mirroring the Node F1 versus Edge F1 gap observed in our study. We instead evaluate the end-to-end RCA capability of multiple real agent frameworks and backbone models directly over telemetry against manually annotated propagation-path ground truth and turn the mined failure modes into a preventive intervention (Section 6).

3 Study Setup

This section establishes the study setup used throughout the paper: the evaluated agents, backbone models, and benchmarks. Fig. 2 gives the study design and the RQ1–RQ3 analysis pipeline; the validation (RQ4–RQ5) follows in Section 6.

3.1 Evaluated Agent Frameworks

To characterize the end-to-end RCA capability of LLM-based agent frameworks and analyze their diagnostic reasoning trajectories, we select nine frameworks spanning two categories (see Section 2.2 for design details). The *general-purpose* group includes four top-ranked investigative agents: ThinkDepth.ai [50], AIQ [38], TaskWeaver [41], and ClaudeCode. The *RCA-specific* group includes five representative designs with publicly available code: OpenRCA [57], mABC [68],

GoS [36], RCLAgent V2 [61], and OpsAgent [37], spanning iterative diagnosis, graph-structured reasoning, multi-agent orchestration, and experience accumulation. All nine use off-the-shelf backbone models; fine-tuning approaches (Section 2.2) are excluded because our study targets the reasoning process of general-purpose LLMs.

3.2 Backbone Models

Our study uses the following three backbone models (USD per 1M input/output tokens, as of 2026-02-15):

- `qwen3.5-plus-2026-02-15` (Qwen): \$0.115/\$0.688.
- `claude-sonnet-4.6` (Sonnet): \$3/\$15.
- `doubao-seed-2.0-pro-2026-02-15` (Doubao): \$0.44/\$2.22.

3.3 Benchmarks

We use three public benchmarks with different topologies and fault granularities.

- RCABench [14]: a microservice RCA benchmark with hierarchical service-to-code-level fault labels; we evaluate on a 500-case sample drawn by simple random sampling without replacement from its 1,430 cases.
- AIOps 2025 [4]: a microservices e-commerce RCA benchmark of 400 incidents over ten core services.
- RCA100 [4]: 103 chaos-drill incidents on the OpenTelemetry Demo; we use the 57-case propagation subset, i.e., the cases whose service-level propagation path, derived from the benchmark's annotated fault chain and the per-case topology, has at least one hop and is supported hop by hop by the released telemetry.

On RCABench, the empirical study (RQ1–RQ3) runs a horizontal comparison of all nine frameworks under the shared Qwen and a vertical comparison of Qwen against Sonnet under ThinkDepth.ai to isolate the backbone-model effect, giving ten framework–backbone configurations (arms). The RQ2–RQ3 trajectory-level analysis focuses on the four general-purpose SQL-call frameworks; the five RCA-specific frameworks retain their native data access and enter the RQ1 baseline comparison without changing the trajectory-analysis cohort. In the intervention and validation phase (RQ4–RQ5), we examine whether the mined failure modes recur beyond the characterization setting and whether DiagGuard, the prototype derived from these modes, improves localization. On the 57-case propagation subset of RCA100, we evaluate baseline and DiagGuard-augmented versions of six frameworks (five RCA-specific plus ThinkDepth.ai) under both Qwen and Doubao, and analyze their failure modes. On the 400 AIOps 2025 incidents, we evaluate ThinkDepth.ai with and without DiagGuard under both backbone models. We further ablate the two defenses using ThinkDepth.ai with Doubao on AIOps 2025.

4 Characterization Methodology

This section details how we turn heterogeneous agent runs into path-grounded, comparable measurements: annotating ground-truth propagation paths and collecting trajectories (Section 4.1), defining outcome and difficulty metrics (Section 4.2), and analyzing trajectory behavior, intent, and failure modes (Section 4.3, Section 4.4).

4.1 Propagation-Path Annotation and Trajectory Collection

Propagation-path annotation. RCABench labels each case's root cause but not the service-level path to the SLO-alarming service, so we annotate this process-level ground truth ourselves. From the injection metadata (targeted service and fault window), the pre-/post-injection traces, metrics, and logs, and the static TrainTicket service-call graph [16], we reconstruct a service-level fault

propagation graph $G^\star = (V^\star, E^\star)$, the union of one or more propagation paths from the root cause to the symptom. Its nodes are the fault-affected services; each directed edge is a propagation step, realized through a logical service call or physical resource sharing and grounded in the abnormal spans, metric shifts, or log signatures within the fault window. This per-case $G^\star$ is the ground truth for Node F1 and Edge F1 (Section 4.2).

Trajectory collection. We collect 5,000 reasoning trajectories. Each run yields thought-action-result triples; a final *compression prompt* then instructs the agent to reason over the collected evidence and report, in a uniform format, a predicted root-cause service and a self-reported causal propagation graph $G = (V, E)$, whose edges need not be query-verified. Both ground the outcome metrics (Section 4.2).

4.2 Metrics Definitions

Trajectory sequence. For each case, the agent's reasoning is an ordered sequence of *rounds* $\mathcal{T} = [\tau_1, \ldots, \tau_n]$, each a triple $\tau_i = (t_i, a_i, r_i)$ of thought, action, and result. The four general-purpose frameworks generate SQL over a shared telemetry interface; the five RCA-specific ones (mABC, OpenRCA, GoS, OpsAgent, RCLAgent V2) retain their native data access.

Reasoning budget. We report each run's per-case reasoning budget as three quantities: *rds* (*effective rounds*: rounds issuing at least one tool call), *ktok* (mean input+output tokens, in thousands), and *$* (those tokens priced at the backbone model's list price; Section 3.2).

Root-cause localization. Each case has a ground-truth root-cause set $R_i^\star$ of one or two services. Acc@1 scores a case correct when the agent's predicted root cause $\hat{r}_i$ lies in $R_i^\star$:

$$\text{Acc@1} = \frac{1}{N} \sum_{i=1}^{N} \mathbf{1}\left(\hat{r}_i \in R_i^\star\right), \tag{1}$$

where N is the number of incidents. For configurations run multiple times (Section 6.3), we also report pass@k, the fraction of incidents with $\hat{r}_i^{(j)} \in R_i^\star$ in at least one of k runs:

$$\text{pass@}k = \frac{1}{N} \sum_{i=1}^{N} \mathbf{1}\left(\exists\, j \in \{1, \ldots, k\} : \hat{r}_i^{(j)} \in R_i^\star\right). \tag{2}$$

Node and edge F1. Let $G^\star = (V^\star, E^\star)$ be the ground-truth service-level fault propagation graph and $G = (V, E)$ the agent's predicted graph from its final result (Section 4.1). We compute

$$\text{Node F1} = \frac{2|V \cap V^\star|}{|V| + |V^\star|}, \tag{3}$$

$$\text{Edge F1} = \frac{2|E \cap E^\star|}{|E| + |E^\star|}. \tag{4}$$

Node F1 measures service-set recovery; Edge F1, directed propagation-link reconstruction. In Table 2, rds, Node F1, and Edge F1 are split by Acc@1 outcome (✓: correct; ×: incorrect; avg: all cases).

Causal-chain depth. We stratify cases by the **causal-chain depth** $d^\star$, the shortest-path length from the root cause to the SLO-alarming service along $G^\star$.

4.3 Behavior and Intent Analysis

The diagnostic goal is to localize a root cause. The services an agent queries at each round reveal whether it is advancing toward that cause along the fault-impact surface ($V^\star$, $E^\star$), and each action's SQL exposes the diagnostic intent behind that advance. We accordingly build two process views

over the trajectories: a *behavior* view, tracking how each round navigates the fault-impact surface and uses surfaced evidence, and an *intent* view, typing each SQL action by its diagnostic intent.

4.3.1 Impact-surface navigation and evidence utilization. We define three per-round labels to characterize each trajectory: a *primary service* that tracks where the investigation stands, a *navigation transition* that captures its direction, and an *evidence-utilization* label that records whether it acts on returned evidence.

Per-round primary service. Each round touches several services, so to track this advance at round granularity we reduce it to a single *primary service*: foremost the round's service nearest a root cause, capturing how far the round advanced. Among equally near services, it is the newly visited one, which marks a new investigation direction, or else the most frequently visited, where the round's investigation has concentrated. Concretely, we draw candidates from the round's own services (its *action services*, or its result services when the round issues no query) and rank them lexicographically by (i) distance from the nearest root cause along $G^\star$, with off-path services taking a large penalty; (ii) unvisited before visited; then (iii) higher cumulative visit count (both relative to earlier rounds). Key (i) sets the round's status (*on-path* if any candidate is on-path, i.e., the round's own queries reach the propagation path, and *off-path* otherwise), and keys (ii) and (iii) break the remaining ties in either case.

Navigation transitions. For each round pair $(i, i{+}1)$ we label a *navigation transition* from the on-path status and distance-to-root of their two primary services, capturing whether the investigation advances toward, stays level with, or moves away from a root cause (full label set in Fig. 3).

Evidence utilization. Independently, an *evidence-utilization* label compares round $i{+}1$'s action services A_{i+1} against the on-path subset of round i's result services R_i, capturing whether the next round acts on the decisive evidence the previous one surfaced (Fig. 3).

4.3.2 Intent taxonomy and sequence patterns. For the intent view, every SQL action receives one intent label and a multi-valued set over six deterministic tag axes (Table 1; Fig. 4). A fixed LLM classifier (`claude-opus-4.8`), following the LLM-as-judge paradigm [21, 29, 73], labels 92,501 SQL actions by reading the compacted trajectory while anchoring on the SQL operation and returned evidence rather than the agent's narrative. The six tag axes are derived deterministically from the SQL text; agreement and classifier accuracy are reported in Section 4.5.

We summarize the intent view in two ways: the *intent distribution*, giving each experiment's frequency over the intents, and *intent n-grams*, capturing contiguous subsequences that surface recurring motifs and repeated-intent loops.

4.4 Failure-Mode Coding and Mining

We code each failed trajectory against the expected evidence for its fault type, yielding three failure families that differ in how the agent relates to decisive evidence (Table 3).

4.4.1 Coding failures against expected evidence. A correct diagnosis must recover sufficient evidence for at least one anomalous path from the observed symptom to a root cause on the fault-impact surface. Using the RQ2 intent and tag vocabulary, we specify each fault's *expected evidence*, the typed queries that would confirm the relevant surface services and dependencies, and read a failed trajectory as a shortfall: evidence the agent omitted or misread, or reasoning untied to evidence.

For each failed trajectory we compare expected evidence against the agent's actual queries and reasoning, drawing on its compacted trajectory, the ground-truth causal context, and raw telemetry when needed.

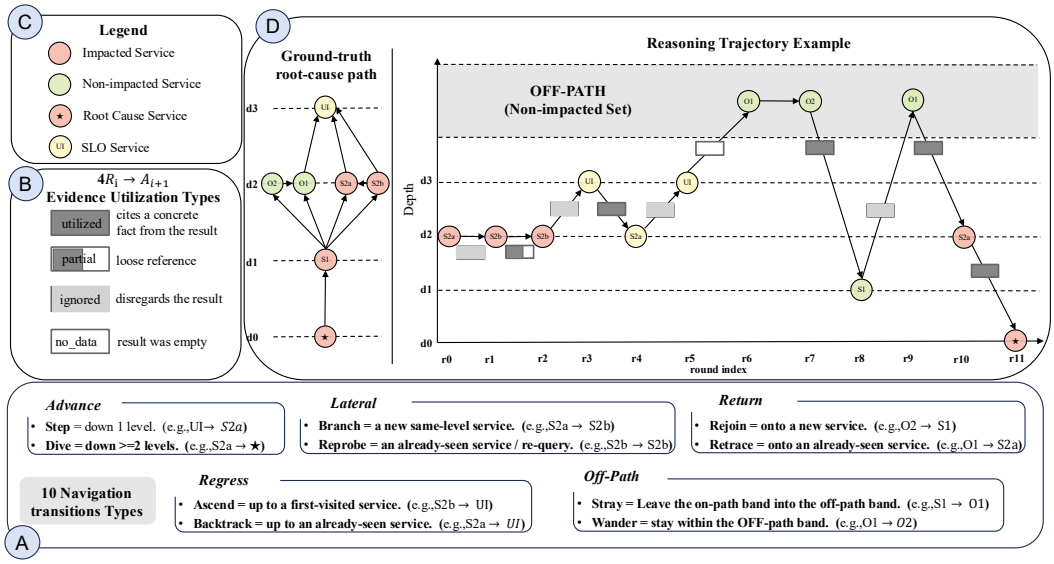


Fig. 3. *Trajectory label vocabularies (RQ2 behavior view).* Each reasoning round is reduced to a *primary service* at some depth on the graph (Section 4.3.1). *Panel **A***: the 10 navigation-transition types, grouped into five families by how the investigation moves relative to a root cause. *Panel **B***: the 4 evidence-utilization types, i.e., whether a round acts on the previous round's on-path evidence. *Panel **C***: node legend for panel D. *Panel **D***: a worked example plotting each round's primary-service depth (y) over round index (x); the grey band is the off-path region and the shaded box under each round marks its evidence-utilization type. These labels are defined in Section 4.3.1 and quantified in Fig. 5a.

```
SELECT time, metric, service_name, attr_k8s_pod_name, value
FROM abnormal_metrics_sum
WHERE service_name = 'ts-order-service'
  AND metric IN ('jvm.memory.used', 'jvm.memory.limit',
       'jvm.cpu.time', 'jvm.thread.count',
       'db.client.connections.usage', 'queueSize')
ORDER BY time ASC, metric
```

intent = resource_probe
window = abnormal · time = timeline
granularity = {pod, service}
resource_family = {cpu, memory, pool, thread}
emission = true · symptom_signal = ∅

Fig. 4. *Worked example of the intent/tag labeling applied to every agent SQL action in the study.* This abnormal-window metric query receives a single SRE *intent* (`resource_probe`) plus six deterministic *tags*; color-matched highlights link each SQL fragment to the tag axis it sets (vocabulary in Table 1).

We code deviations into three families (Table 3): *protocol omission* (`OMIT`), evidence reachable in telemetry but never queried; *semantic misread* (`MIS`), retrieved evidence misinterpreted or scoped to the wrong dependency; and *general reasoning* (`GEN`), a failure not tied to a single evidence unit.

Two authors jointly develop the codebook through open coding of failed trajectories, letting the recurring failure codes emerge and refining them until the codebook stabilizes; an LLM (`claude-opus-4.8`) assists evidence retrieval [67, 73], but every code is verified by hand against the cited SQL, returned evidence, and the TrainTicket service-call graph [16]. Using the fixed codebook, the two authors independently code all 154 failed ThinkDepth.ai trajectories (50 Sonnet, 104 Qwen); agreement is reported in Section 4.5.

Table 1. SRE diagnostic-intent and tag vocabulary used to label each agent action. The eleven intents (left) classify the diagnostic purpose of each SQL query; the six tag axes (right) are derived deterministically from the SQL text and describe its operational scope.

Axis	Family	Name	Definition
Intent	Symptom	`symptom_scan`	Compares service-level symptoms, including errors, latency, status codes, payload shape, and request volume.
		`log_inspect`	Inspects log records for errors, exceptions, stack traces, abnormal volume, and other textual symptoms.
		`text_search`	Searches telemetry text for service names, error patterns, fault keywords, and suspicious literals.
	Topology	`trace_follow`	Follows trace IDs, spans, and parent-child links to localize propagation along service calls.
		`callgraph`	Queries caller-callee relations, span kinds, and call ratios to recover dependency direction and structure.
		`network_flow`	Inspects request flow, failures, connectivity, packet/drop signals, and service-to-service traffic.
	Resource	`resource_probe`	Tests resource hypotheses with metrics for named services, pods, nodes, JVMs, DBs, or runtimes.
		`metric_dump`	Retrieves broad metric values or snapshots to scan resource state before narrowing a hypothesis.
	Liveness	`lifecycle_probe`	Checks lifecycle/emission signals such as restarts, silence, span export, pod state, and JVM state.
	Inventory	`metric_enum`	Enumerates metric names, schemas, coverage, resource attributes, and measurement inventory.
		`service_enum`	Enumerates services, pods, nodes, tables, topology entities, and telemetry inventory.
Tag	Derived	`granularity`	Target level over which evidence is grouped, e.g., global, node, pod, service, endpoint, or trace.
		`window`	Telemetry window compared by the query, e.g., abnormal, normal, or normal-abnormal contrast.
		`time`	Temporal shape requested by the query, e.g., snapshot, timeline, or onset analysis.
		`emission`	Whether the query references telemetry emission, silence, exporter, queue, or liveness fields.
		`resource_family`	Resource axis referenced by the query, e.g., CPU, memory, GC, pool, thread, or disk.
		`symptom_signal`	Symptom signal referenced by the query, e.g., error, status code, latency, payload, or volume.

4.5 Annotation Reliability

We assess the reliability of propagation-graph annotations, SRE intent labels, and failure-mode annotations in RQ3 and RQ4. For propagation graphs, intent labels, and RQ4 annotations, we follow a shared review protocol: one annotator produces the initial annotations with algorithmic or LLM assistance, as appropriate; two reviewers independently accept or reject each annotation; and a senior author adjudicates disagreements. The agreement rates and Cohen's κ reported for these three tasks measure agreement between the reviewers' binary acceptance decisions.

For all 500 propagation graphs, the reviewers agree on 97.4% of decisions (κ=0.59), with individual acceptance rates of 96.8% and 96.6%. For a stratified sample of 1,000 intent labels, they agree on 98% (κ=0.73); after adjudication, 96% of the original labels are accepted. For a stratified sample of 180 RQ4 failure-mode annotations (15 per framework–backbone configuration), they agree on 97.2% (κ=0.53); after adjudication, 95.6% of the original annotations are accepted.

For RQ3, after the codebook developed through open coding (Section 4.4.1) stabilizes, two authors independently recode all 154 incorrectly localized trajectories using the codebook. They assign identical code sets to 144 trajectories (93.5%), measuring exact agreement at the trajectory level.

5 Characterization Results

This section characterizes RCA agents along three deepening levels: RQ1 establishes outcome and cost, RQ2 explains the outcome gaps through behavior and intent, and RQ3 distills wrong diagnoses into recurring failure modes, which Section 6 later turns into an intervention.

5.1 RQ1: Effectiveness and Cost

Framework and Model Comparison. Table 2 compares agents along two axes. Across frameworks, effectiveness is shaped by investigative architecture: the agents that run an open-ended, adaptive investigation, requerying telemetry as evidence emerges (ThinkDepth.ai 79.2%, AIQ 77.6%, ClaudeCode 79.6%), hold the top band in a narrow range, while the more constrained designs

Table 2. **RQ1**: outcome-level comparison of all ten framework–backbone configurations on RCABench. Edge F1 trails Node F1 by a wide margin, and Acc@1 falls as the causal chain deepens, a decline the Sonnet row largely escapes. Metrics and the ✓/× outcome split defined in Section 4.2; Acc@1 and depth columns in %, F1 in [0, 1]. Costs use the model prices in Section 3.2.

			Node F1			Edge F1			Rounds (**rds**)			Budget / case		Acc@1 by depth $d^\star$			
Backbone model	**Framework**	Acc@1	✓	×	avg	✓	×	avg	✓	×	avg	*ktok*	$	$d^\star$=2	$d^\star$=3	$d^\star$=4	$d^\star$=5
	mABC	42.6	0.49	0.35	0.41	0.15	0.05	0.09	–	–	–	163	0.04	53.3	36.4	40.4	42.9
	OpsAgent	45.8	0.81	0.52	0.65	0.61	0.26	0.42	–	–	–	83	0.01	42.1	49.6	44.7	42.9
	OpenRCA	46.6	0.57	0.48	0.52	0.36	0.16	0.25	6.1	6.7	6.4	364	0.05	58.6	46.9	30.9	19.0
	GoS	48.2	0.77	0.51	0.64	0.57	0.25	0.40	–	–	–	92	0.01	57.9	50.0	33.0	19.0
Qwen	RCLAgent V2	53.4	0.74	0.42	0.59	0.51	0.16	0.35	108.9	175.6	140.0	773	0.13	56.6	55.7	43.6	52.4
	TaskWeaver	65.2	0.78	0.55	0.70	0.58	0.26	0.47	13.2	14.5	13.7	672	0.09	79.6	68.9	39.4	33.3
	AIQ	77.6	0.81	0.63	0.77	0.61	0.35	0.55	50.0	55.1	51.1	1516	0.19	86.8	82.0	59.6	42.9
	ClaudeCode	79.6	0.81	0.58	0.76	0.65	0.28	0.57	51.3	62.7	53.6	1512	0.18	89.5	79.8	68.1	57.1
	ThinkDepth.ai	79.2	0.83	0.59	0.78	0.66	0.33	0.59	43.1	50.7	44.7	1744	0.21	85.5	82.0	66.0	57.1
Sonnet	ThinkDepth.ai	90.0	0.82	0.56	0.79	0.67	0.37	0.64	30.5	38.2	31.2	1465	4.64	88.2	88.6	96.8	85.7

trail: TaskWeaver (65.2%) follows a fixed plan; RCLAgent V2 (53.4%) spins up one sub-agent per trace span and checks each in isolation; GoS (48.2%) fixes its hypotheses up front and, capped at three retrieval steps, tests only the most credible one; OpenRCA (46.6%) commits as soon as it can emit an answer; OpsAgent (45.8%) issues no telemetry queries of its own, instead reasoning over a pre-extracted feature set; and mABC (42.6%) does not iterate over evidence at all. Notably, none of the top agents was built for RCA, yet every agent built for it trails them: open-ended investigative frameworks show a clear advantage on end-to-end RCA over those limited by a fixed control flow or constrained data access.

Across backbone models, the Sonnet ThinkDepth.ai arm reaches 90.0% Acc@1, a 10.8-pp gain over Qwen. The top-band frameworks span only 2.0 pp under the same model (77.6%–79.6%), while the model swap yields five times that gap, suggesting the backbone model as the dominant lever.

Finding 1

For end-to-end diagnostic tasks, adaptive, minimally constrained frameworks outperform more tightly orchestrated designs; among the top-band frameworks, the backbone model is the dominant lever.

Causal-Chain Depth and Reconstruction Granularity. Causal-chain depth serves as a proxy for diagnostic difficulty: a longer chain demands more investigative steps and more evidence to reach the root cause. Empirically, accuracy declines accordingly. Nearly all Qwen frameworks lose ground from depth 2 to depth 5 (ClaudeCode 89.5 → 57.1%, AIQ 86.8 → 42.9%); on the matched ThinkDepth.ai pair, Qwen falls likewise (85.5 → 57.1%) while Sonnet holds flat (85.7–96.8%), widening the backbone-model gap on the hardest cases.

Acc@1 and surface reconstruction are positively coupled across Table 2, yet the backbone-model swap lifts localization without lifting the reconstruction ceiling. Weaker agents do not merely select the wrong final service: they reconstruct less of the fault-impact surface even on their correctly localized cases (✓ Node F1 rises from 0.49 for mABC to 0.83 for ThinkDepth.ai, ✓ Edge F1 from 0.15 to 0.66). Even correct localizations, however, rest on partial causal maps: ✓ Edge F1 never exceeds 0.67, and the model swap that raises Acc@1 by 10.8 pp leaves this ceiling nearly untouched (Node F1 0.82 vs. 0.83, Edge F1 0.67 vs. 0.66). In every row, Edge F1 falls well below Node F1 (mABC: 0.41 vs.

0.09; ClaudeCode: 0.76 vs. 0.57), and failure widens this imbalance: from correct to incorrect runs, Node F1 drops by 16–43% while Edge F1 drops by 43–69%, so failed runs still detect most anomalous services yet no longer chain them into a propagation path. Identifying anomalous nodes requires only detecting aberrant telemetry; recovering edges requires inferring how the fault propagates between services, making edge reconstruction the harder and more discriminative capability.

Finding 2

Accuracy degrades with causal-chain depth, a decline the stronger backbone model largely escapes. Reconstructing the fault-impact surface is harder than selecting the correct final answer and remains partial even when localization succeeds; within the reconstruction, recovering directed propagation edges proves more demanding than identifying anomalous nodes.

Reasoning Budget and Diagnostic Outcome. Budget scales with investigative openness: the top-band agents sustain 45–54 effective rounds, over 1500 ktok, and roughly $0.2 per case, whereas the constrained agents stop far sooner (under 700 ktok, at most $0.09). RCLAgent V2 is the exception: it runs a separate sub-agent for each trace span, so its round counts grow with trace size rather than with evidence-driven, open-ended search. Within ThinkDepth.ai, Sonnet stops in fewer rounds than Qwen, locking onto decisive evidence quickly where Qwen enumerates more broadly, though Sonnet's higher per-token price still dominates per-case cost.

Yet one pattern holds across the sequentially iterating agents: incorrect cases run more rounds than correct ones, by up to 11.4 rounds. The extra rounds buy no partial credit either: incorrect runs trail correct ones on both F1 metrics, so the added budget reflects unproductive search rather than evidence accumulation.

Finding 3

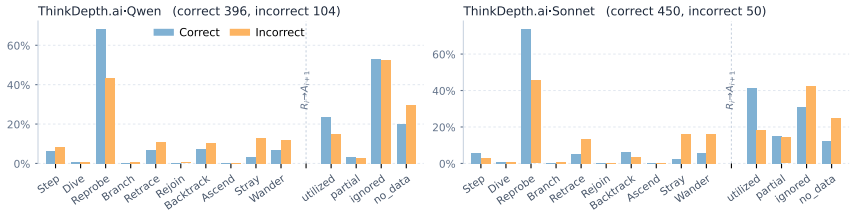
Reasoning budget varies across frameworks and backbone models, but within a framework more rounds do not yield better diagnoses: failed runs consume more rounds than successful ones, as unproductive search inflates cost without improving outcomes.

5.2 RQ2: Agent Behavior

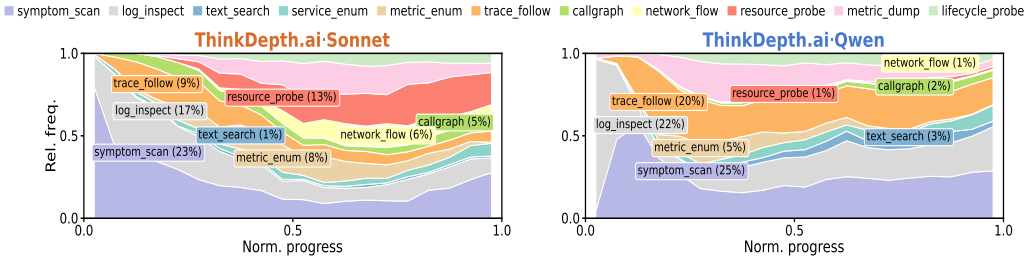
Having established outcome and cost differences, we now examine the diagnostic process behind them. Because RQ2–RQ3 analyze SQL-call trajectories, they focus on the four general-purpose frameworks and exclude the five RCA-specific frameworks, which retain their native data access (Section 4.2). We center on ThinkDepth.ai's Qwen and Sonnet arms, the only matched pair; the Qwen arm is representative because all Qwen SQL-call frameworks show similar behavior and intent patterns.

Surface Navigation and Evidence Grounding. Correct runs ground each round in retrieved evidence; incorrect runs drift off-path or ignore decisive clues (Fig. 5a). Specifically, correct runs lean on `Reprobe`, re-querying on-path services at the same depth (higher than incorrect runs in both arms), whereas incorrect runs oscillate on and off the surface (higher `Stray`/`Wander` and `Retrace`). The two arms navigate in similar proportions.

Evidence grounding, not navigation volume, is where the two backbone models diverge. Correct runs act on the evidence they retrieve (higher `utilized`), whereas incorrect runs more often hit empty queries (higher `no_data`) and, in the Sonnet arm, ignore results (higher `ignored`). Sonnet acts on the prior result (`utilized` or `partial`) on 54.3% of rounds versus Qwen's 24.9%. In effect, Sonnet's backtracks and advances are evidence-guided confirmations, while Qwen swings between shallow and deep nodes regardless of the returned evidence.



(a) **Navigation and evidence-utilization label shares**, correct (blue) vs. incorrect (orange) runs per arm. In the navigation block (left), the correct–incorrect contrast is consistent across both arms; in the evidence-utilization block (right, after the dashed line), the two backbone models diverge sharply. Labels defined in Fig. 3; bars macro-averaged per trajectory.



(b) **SRE diagnostic-intent composition** over normalized trajectory progress. Both arms share a symptom-dominated opening phase; band shapes diverge from mid-trajectory onward, with the Sonnet arm shifting weight toward intent categories largely absent in Qwen. Band thickness is each intent's relative frequency (colors: the 11 intents of Table 1); in-plot labels give overall shares.

Fig. 5. **RQ2**: behavior and intent analysis. What separates correct from incorrect runs (a) and what drives the backbone-model gap in diagnostic strategy (b).

Finding 4

Success and failure come down to process discipline: a successful diagnosis stays within the fault-impact surface and uses the evidence it retrieves, while a failing one wanders off or ignores decisive evidence.

Intent Repertoire and Fault-Type Sensitivity. The two backbone models deploy qualitatively different diagnostic strategies, with Sonnet pivoting mid-trajectory to resource- and topology-oriented intents where Qwen persists in shallow trace and log loops (Fig. 5b). Over the full eleven-intent profile, the Sonnet arm is the least similar to every Qwen arm (cosine similarity 0.907–0.928), while the four Qwen frameworks cluster tightly (0.947–0.989).

Both arms open with a shared symptom-scanning phase, after which their strategies diverge. Qwen stays trace- and log-bound: as the trajectory advances it relies increasingly on fuzzy `text_search` over log and trace text and on `trace_follow` span enumeration, and although it inventories metrics via `metric_enum` and `metric_dump`, it rarely drills into them. Sonnet runs the same early symptom/log/trace survey but then pivots in mid-trajectory to exploring the metric query space (`metric_enum`), recovering dependency structure with `callgraph`, and drilling named resources and flows via `resource_probe` (used in 82.8% of its trajectories vs. 16.6% for Qwen) and `network_flow`. This maps onto per-fault accuracy: Qwen trails Sonnet most on categories requiring

Table 3. **RQ3**: three evidence-handling failure families. Each family addresses a different relationship between the agent and decisive evidence: never gathered (`OMIT`), gathered but misread (`MIS`), or overridden by ungrounded reasoning (`GEN`).

Family	Mode	Definition	Consequence	Sonnet	Qwen
Protocol omissions	`OMIT1` Baseline gap	No comparison against a pre-incident healthy baseline.	Pre-existing signal misjudged as incident evidence.	26.0%	51.9%
	`OMIT2` Edge untested	No span-level test on the claimed caller–callee edge.	Propagation edge asserted without span-level evidence.	54.0%	68.3%
	`OMIT3` Resource gap	No resource or lifecycle probe using concrete entity names.	Decisive resource or lifecycle signal remains unseen.	34.0%	63.5%
	`OMIT4` Ranking gap	No peer ranking to check if the signal is unique.	Collateral spike shared across peers treated as uniquely causal.	2.0%	25.0%
	`OMIT5` Granularity gap	No drill-down to pod, endpoint, edge, or metric level.	Specific failing unit remains unidentified at fine granularity.	18.0%	16.3%
	`OMIT6` Timing gap	No check that proposed cause precedes the symptom.	Causal claim lacks temporal corroboration.	4.0%	8.7%
Semantic misreads	`MIS1` Silence-as-health	Silence, flatness, or missing data read as healthy.	Suppressed or unreported fault incorrectly cleared as healthy.	30.0%	44.2%
	`MIS2` Salience bias	Loudest anomaly signal over-trusted without cause–symptom distinction.	Noisy victim service selected as cause over quieter culprit.	42.0%	32.7%
	`MIS3` Semantics misread	Telemetry read with the wrong service-side semantics.	Fault direction reversed between caller and callee sides.	86.0%	61.5%
General reasoning	`GEN1` Anchor lock	Early hypothesis retained despite later conflicting evidence.	Contradictory evidence discounted in the final answer.	28.0%	34.6%
	`GEN2` Schema-out claim	Causal state fabricated outside the prescribed set of states.	Unvalidatable out-of-schema state returned as root cause.	82.0%	20.2%
	`GEN3` Friction stop	Evidence channel dropped after tool error or empty return.	Decisive evidence from that channel remains ungathered.	0%	12.5%

Note. Percentages report within-model failure-pool prevalence (n=50 Sonnet, n=104 Qwen; all from ThinkDepth.ai); subcategories may overlap and do not sum to 100%.

named-resource metrics (Resource 75% vs. 94%, Code (JVM) 66% vs. 93%), while Network nearly closes (82% vs. 89%) and HTTP is tied (88% vs. 87%). Qwen further perseverates in shallow loops that Sonnet avoids: intent 5-grams such as sustained trace-following (`trace_follow`[5], 0.96 per trajectory) and `log_inspect/symptom_scan` oscillation recur far more in the Qwen arm than in Sonnet ($\leq$0.12 per trajectory), the sequence-level signature of its narrow repertoire.

Finding 5

A richer diagnostic intent repertoire corresponds to broader, SRE-style search, while a narrow one keeps the agent in shallow loops over a limited slice of the evidence, producing fault-type-specific accuracy gaps that concentrate on the types requiring deeper evidence.

5.3 RQ3: Failure Taxonomy

The behavioral patterns above show *how* runs diverge; we now ask *why* wrong diagnoses occur. Wrong diagnoses reduce to three evidence-handling failure families that differ in how the agent relates to decisive evidence (Table 3).

Failure families and cross-model patterns. The general-reasoning failures in Table 3 are untied to any single signal: a model that cannot aggregate scattered evidence or drill deep enough to discard distractors locks onto an early anchor (`GEN1`); an overconfident one fills gaps by fabricating an out-of-schema cause (`GEN2`); and one without the persistence to retry a failed or empty query abandons the evidence channel altogether (`GEN3`).

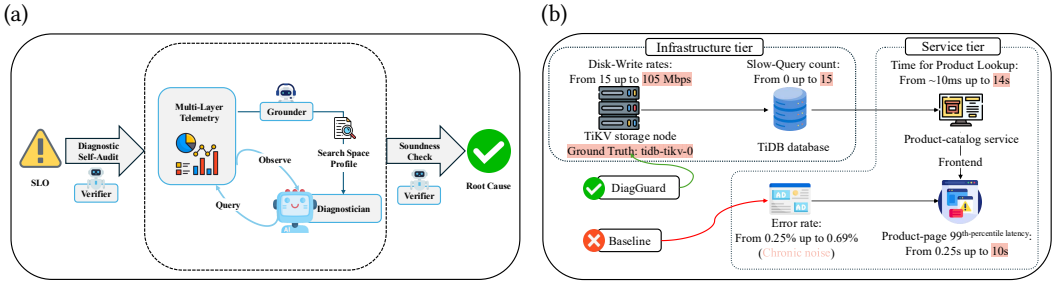


Fig. 6. (a) **The DiagGuard prototype.** Two answer-agnostic defenses derived from the failure taxonomy (Table 3) augment the Diagnostician at three temporal junctures. The *Verifier* runs a diagnostic self-audit pre-investigation and a soundness check pre-commitment; the *Grounder* supplies a search-space profile before the first query. (b) **Motivating incident (Doubao / AIOps 2025).** An injected disk-I/O fault on the TiKV storage node `tidb-tikv-0` surfaces only as downstream tail latency; red-highlighted values mark abnormal-vs-normal shifts. The baseline (red cross) anchors on the chronically noisy Ad service and never leaves the service tier, whereas DiagGuard (green check) descends through the infrastructure tier to the true root cause `tidb-tikv-0` (Section 6).

The evidence-anchored families track the two backbone models' reasoning posture. Qwen fails predominantly by omission, Sonnet by misreading. Qwen has less working knowledge of the RCA problem domain, so it stays redundantly confined to the most superficial, easily found signals (`OMIT1`, `OMIT3`, `OMIT4`) and rarely redirects its exploration from the evidence already gathered (`GEN1`). Sonnet has a stronger ability to hypothesize new investigation directions from the evidence, but its strong associative reach sometimes errs through hallucination (`GEN2`), and its synthesis often falls short of pinning down the crux among contradictory signals (`MIS2`, `MIS3`).

Finding 6

Agent failures include three recurring families: decisive evidence is never gathered, is gathered but misread, or is overridden by ungrounded reasoning.

Whether distilling these recurring modes into defenses can improve localization beyond the characterization setting remains to be tested.

6 Intervention and Validation

This section examines whether the RQ3 failure taxonomy can guide diagnostic interventions that generalize beyond the original characterization setting. We introduce DiagGuard (Section 6.1) and evaluate its localization gains across datasets, backbone models, and frameworks (Section 6.2). We also examine whether the catalogued failure modes recur, how their prevalence changes under intervention, and whether framework-associated differences persist. Finally, component ablations assess the individual and combined contributions of DiagGuard's two defenses (Section 6.3).

A held-out incident illustrates the intervention (Fig. 6b). A disk-I/O fault on `tidb-tikv-0` manifests as downstream tail latency. The baseline ThinkDepth.ai agent anchors on a chronically noisy service and fails to investigate the storage tier, exhibiting several catalogued failure modes (`MIS2`, `OMIT1`, `OMIT3`, `OMIT5`, `OMIT6`, `GEN3`; Table 3). With DiagGuard, the agent extends its investigation to the infrastructure tier and correctly identifies the faulty storage node.

6.1 DiagGuard Design

The taxonomy of Section 5.3 doubles as a design specification: its three failure families jointly inform the design of two answer-agnostic defenses. **DiagGuard** wraps a diagnostic framework's reasoning loop in these defenses at three temporal junctures (Fig. 6a), drawing on agentic grounding [8] and inference-time verification [52] with rules traced to the mined failure codes.

The core is the *Diagnostician*, the unmodified diagnostic framework, whose standalone run serves as the baseline in Section 6.2. The **Verifier** injects a diagnostic self-audit before the Diagnostician begins its investigation, preventing the `OMIT` and `MIS` families (evidence routinely skipped or misread) from going unchecked; when the Diagnostician drafts a verdict, the Verifier injects a soundness check that curbs the `GEN` family by requiring the agent to reconcile its answer with the evidence already gathered. The **Grounder** interposes a data-grounding layer before the Diagnostician writes its first query, replacing passive schema inspection with a search-space profile that surfaces overlooked diagnostic axes (`OMIT`) and anchors the agent's queries in actual data contents (`MIS`). Both defenses are integrated into each framework through prompts and tools, with rules fixed across validation settings.

6.2 RQ4: Intervention Effectiveness and Failure-Mode Recurrence

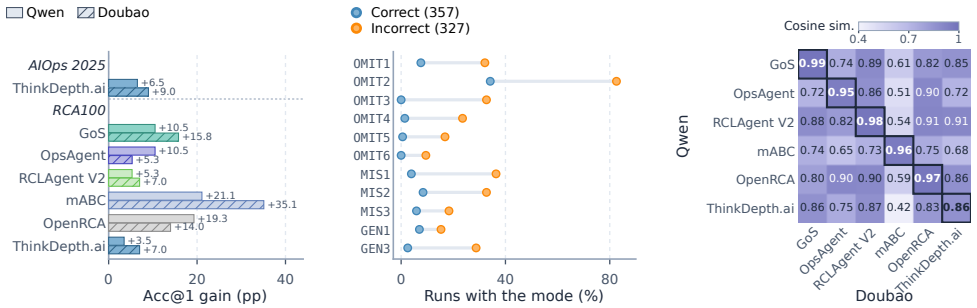
We evaluate whether DiagGuard improves localization beyond the characterization setting and whether the catalogued failure modes recur on RCA100. The evaluation covers ThinkDepth.ai on AIOps 2025 and six frameworks on RCA100, each under Qwen and Doubao (fourteen configurations; Fig. 7a). We apply the fixed RQ3 codebook to all 1,368 RCA100 runs (684 baseline and 684 DiagGuard) to examine mode recurrence, prevalence changes, and framework-associated failure profiles.

Generalization of intervention effectiveness. With fixed intervention rules, DiagGuard improves Acc@1 in all fourteen configurations. On AIOps 2025, ThinkDepth.ai gains 6.5 and 9.0 percentage points under Qwen and Doubao, respectively. Across the twelve RCA100 configurations, the median gain is 10.5 percentage points (range: 3.5–35.1). The gains span graph-structured reasoning (GoS, +10.5/+15.8 pp under Qwen/Doubao), experience accumulation (OpsAgent, +10.5/+5.3), and general-purpose investigation (ThinkDepth.ai, +3.5/+7.0). Multi-agent orchestration also benefits: mABC shows the largest gains (+21.1/+35.1), from baseline accuracies of 35.1% and 3.5%. On RCA100, Node F1 and Edge F1 for the final reported propagation graphs each improve in eleven of twelve configurations. Their median increases across all twelve configurations are 0.06 and 0.10, respectively.

Finding 7

With fixed intervention rules, DiagGuard improves localization accuracy in all fourteen configurations across datasets and backbone models, spanning diagnostic frameworks from constrained search or data-acquisition workflows to end-to-end, open-ended investigation.

Failure-mode recurrence on RCA100. All eleven evaluated modes (excluding `GEN2`, which framework constraints make incomparable) recur in baseline RCA100 runs, supporting their applicability beyond the original setting. The codes describe process defects that can occur even when localization is correct. Nevertheless, every mode is more prevalent in incorrect than correct baseline runs (Fig. 7b). Incorrect and correct baseline runs exhibit 3.29 and 0.71 distinct modes on average, respectively. Under DiagGuard, the corresponding means are 2.82 and 0.63, showing that the association with incorrect localization persists.



(a) Acc@1 improvements over corresponding baselines

(b) Failure-mode prevalence by baseline run outcome

(c) Failure-mode similarity across frameworks and backbones

Fig. 7. **RQ4**: accuracy gains with DiagGuard and patterns of failure modes. (a) Acc@1 improvement over baseline, in percentage points. Solid bars denote Qwen; hatched bars denote Doubao. All configurations improve with DiagGuard. (b) Percentage of correct and incorrect baseline runs exhibiting each failure mode, calculated separately for the two outcome groups. Every mode is more prevalent among incorrect runs. (c) Cross-backbone similarity of framework failure profiles. Each profile contains the prevalence of each failure mode among that framework's incorrect baseline runs. Rows represent frameworks using Qwen; columns represent frameworks using Doubao. Cell values indicate cosine similarity, with diagonal cells comparing the same framework across backbones. Failure-mode prevalence is more similar across backbones within the same framework than across frameworks. Failure modes are defined in Table 3.

Changes in failure-mode prevalence. Under DiagGuard, the mean number of distinct modes per run decreases from 1.94 to 1.39, with reductions in all twelve RCA100 configurations. Across 684 runs per condition, seven modes show statistically significant prevalence reductions. The largest decreases occur in `OMIT2` (57.3% to 44.7%) and `GEN3` (15.1% to 4.7%). `MIS3` (Semantics misread) does not decrease significantly under DiagGuard (11.8% to 13.6%). One possible explanation is that the intervention improves evidence acquisition without fully addressing errors in interpreting the retrieved evidence.

Framework-associated failure profiles after intervention. We next examine whether framework-associated failure patterns persist after intervention. For each framework–backbone configuration, we compute an eleven-mode prevalence vector over its incorrect runs and compare vectors using cosine similarity. Each entry is the fraction of incorrect runs exhibiting the corresponding mode. Mean cross-backbone similarity is higher within the same framework than across frameworks under both baseline (0.950 vs. 0.767; Fig. 7c) and DiagGuard (0.897 vs. 0.720). These comparisons indicate that framework-associated differences remain observable across the two evaluated backbones after intervention.

Finding 8

All eleven evaluated modes recur on RCA100, and DiagGuard significantly reduces the prevalence of seven. Under both conditions, failure-mode profiles are more similar, on average, across backbones within the same framework than across frameworks.

6.3 RQ5: Component Contribution

Table 4 ablates the two defenses on AIOps 2025 (Doubao, 400 incidents, five runs). Each helps on its own by a comparable margin: the Grounder alone (*w/o Verifier*) raises Acc@1 by +4.5 and

Table 4. **RQ5**: component ablation of DiagGuard on AIOps 2025 (Doubao, 400 incidents). The Δ column shows near-additive gains that do not track the budget columns (rds, ktok, $): the costliest arm gains least, and the full stack gains most while spending less than it. Acc@1 is mean±sd over five runs.

	Acc@1		pass@3	pass@5	Budget / incident		
Configuration	%	Δ	%	%	*rds*	*ktok*	*$*
Baseline (w/o both)	$43.5_{\pm 0.6}$	—	56.9	62.3	14.1	176.8	0.09
w/o Grounder	$47.6_{\pm 1.6}$	+4.1	61.3	65.8	19.2	343.8	0.17
w/o Verifier	$48.0_{\pm 0.9}$	+4.5	62.0	67.3	16.3	212.1	0.10
DiagGuard (full)	$\mathbf{52.5}_{\pm 1.1}$	**+9.0**	**67.1**	**73.0**	18.7	318.5	0.16

the Verifier alone (*w/o Grounder*) by +4.1, with the full DiagGuard reaching +9.0 and the same ordering at pass@3 and pass@5. Both defenses improve accuracy individually, and combining them yields a larger gain than either alone, approximately matching the sum of their individual Acc@1 gains.

7 Threats to Validity

External validity. Findings from a single microservice system may not generalize. We therefore evaluate on two additional systems with different topologies (AIOps 2025 and RCA100) and compare across backbone models and frameworks; the failure modes catalogued on RCABench recur on RCA100, suggesting that the observed patterns are not system-specific. All three systems are open source and may appear in pretraining data, but each framework and DiagGuard comparison holds the backbone fixed, so such leakage affects compared arms equally.

Internal validity. We include all publicly available state-of-the-art RCA frameworks we could execute, but some components are unreleased or incompatible with API-served backbones (e.g., OpsAgent's PPO self-evolution), which may understate those frameworks. To ensure fair comparison, all frameworks run on the same cases under the same backbone with the same metrics, while each RCA-specific framework retains its native data access. Another threat is potential bugs in our implementation; we address this through extensive testing and by releasing all code, data, and scripts for independent verification.

Construct validity. Several labels require human judgment, which may introduce bias. Two annotators independently label each task, with a senior annotator resolving disagreements (93.5–98% agreement; Section 4.5); we release the annotation records and codebook for reproduction. Additionally, standard metrics such as Top@K and MRR may not fully capture the practical interpretability needs of an operator; we acknowledge this limitation and scope our evaluation to automated localization accuracy.

Conclusion validity. A single run may not capture run-to-run variation. We repeat the AIOps 2025 ablation five times, reporting mean±sd and pass@k (Table 4), and draw conclusions from the consistent gain across all fourteen configurations rather than from any single comparison.

8 Conclusion

We presented a process-level study of how LLM agents reach a root-cause diagnosis, not merely whether it is correct. Using manually validated fault propagation paths, we analyzed 5,000 trajectories from ten framework–backbone configurations. Correct localization often masks incomplete reconstruction of fault propagation, particularly the links between affected services. Detailed analysis of four frameworks with comparable SQL actions shows that successful investigations tend

to stay on the fault-impact surface and use the evidence they retrieve, while failed investigations more often drift off the surface or leave decisive evidence unused. A broader range of diagnostic intents is associated with deeper investigation. We identify three recurring failure families: protocol omissions, semantic misreads, and general-reasoning failures.

DiagGuard, a prototype developed from these failure modes, improves localization accuracy in all fourteen tested configurations across two additional benchmarks, two backbone models, and six frameworks, with gains of 3.5 to 35.1 percentage points. The evaluated failure modes recur across settings, and DiagGuard reduces their mean count per run. Even after intervention, the distributions of failure modes remain, on average, more similar across backbone models within the same framework than across frameworks. Together, these findings support assessing fault propagation paths and evidence use alongside localization accuracy, and using recurring failures to guide improvements to individual frameworks. Future work will use annotated paths and failure labels to guide agent training and extend the evaluation to production incidents.

References

[1] Toufique Ahmed, Supriyo Ghosh, Chetan Bansal, Thomas Zimmermann, Xuchao Zhang, and Saravan Rajmohan. 2023. Recommending root-cause and mitigation steps for cloud incidents using large language models. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, 1737–1749.

[2] Anthropic. 2024. Building Effective Agents. https://www.anthropic.com/research/building-effective-agents. Accessed: 2026.

[3] Islem Bouzenia and Michael Pradel. 2025. Understanding software engineering agents: A study of thought-action-result trajectories. *arXiv preprint arXiv:2506.18824* (2025).

[4] Yuanhong Cai, Xiaohui Nie, Kanglin Yin, Changhua Pei, Yongqian Sun, Shenglin Zhang, Haibin Liu, Guiyang Liu, Xidao Wen, Fang Situ, et al. 2026. A Multi-Dataset Benchmark for Evaluating LLM Agents in Microservice Failure Diagnosis. *arXiv preprint arXiv:2606.29193* (2026).

[5] Sarthak Chakraborty, Shaddy Garg, Shubham Agarwal, Ayush Chauhan, and Shiv Kumar Saini. 2023. CausIL: Causal Graph for Instance Level Microservice Data. In *Proceedings of the ACM Web Conference 2023 (WWW)*. ACM, 2905–2915. https://doi.org/10.1145/3543507.3583274

[6] Yinfang Chen, Jiaqi Pan, Jackson Clark, Yiming Su, Noah Zheutlin, Bhavya Bhavya, Rohan R. Arora, Yu Deng, Saurabh Jha, and Tianyin Xu. 2025. STRATUS: A Multi-agent System for Autonomous Reliability Engineering of Modern Clouds. In *Advances in Neural Information Processing Systems*. arXiv:2506.02009 [cs.DC] https://neurips.cc/virtual/2025/poster/116834

[7] Yinfang Chen, Huaibing Xie, Minghua Ma, Yu Kang, Xin Gao, Liu Shi, Yunjie Cao, Xuedong Gao, Hao Fan, Ming Wen, et al. 2024. Automatic root cause analysis via large language models for cloud incidents. In *Proceedings of the Nineteenth European Conference on Computer Systems*. 674–688.

[8] Minghang Deng, Ashwin Ramachandran, Canwen Xu, Lanxiang Hu, Zhewei Yao, Anupam Datta, and Hao Zhang. 2025. Reforce: A Text-to-SQL agent with self-refinement, format restriction, and column exploration. In *ICLR 2025 workshop: verifAI: AI verification in the wild*.

[9] Zhirui Deng, Zhicheng Dou, Yutao Zhu, Ji-Rong Wen, Ruibin Xiong, Mang Wang, and Weipeng Chen. 2024. From novice to expert: LLM agent policy optimization via step-wise reinforcement learning. *arXiv preprint arXiv:2411.03817* (2024).

[10] Mingxuan Du, Benfeng Xu, Chiwei Zhu, Xiaorui Wang, and Zhendong Mao. 2025. Deepresearch bench: A comprehensive benchmark for deep research agents. *arXiv preprint arXiv:2506.11763* (2025).

[11] Will Epperson, Gagan Bansal, Victor C Dibia, Adam Fourney, Jack Gerrits, Erkang Zhu, and Saleema Amershi. 2025. Interactive debugging and steering of multi-agent ai systems. In *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems*. 1–15.

[12] Aoyang Fang, Haowen Yang, Haoze Dong, Qisheng Lu, Junjielong Xu, and Pinjia He. 2025. A Goal-Driven Survey on Root Cause Analysis. *arXiv preprint arXiv:2510.19593* (2025).

[13] Aoyang Fang, Yifan Yang, Jin'ao Shang, Qisheng Lu, Junjielong Xu, Rui Wang, Songhan Zhang, Yuzhong Zhang, Boxi Yu, and Pinjia He. 2026. OpenRCA 2.0: From Outcome Labels to Causal Process Supervision. *arXiv preprint arXiv:2606.27154* (2026).

[14] Aoyang Fang, Songhan Zhang, Yifan Yang, Haotong Wu, Junjielong Xu, Xuyang Wang, Rui Wang, Manyi Wang, Qisheng Lu, and Pinjia He. 2025. Rethinking the Evaluation of Microservice RCA with a Fault Propagation-Aware Benchmark. *arXiv preprint arXiv:2510.04711* (2025).

[15] Nan Fu, Guang Cheng, Yue Teng, Guangye Dai, Shui Yu, and Zihan Chen. 2025. Intelligent Root Cause Localization in MicroService Systems: A Survey and New Perspectives. *Comput. Surveys* 57, 12, Article 325 (2025). https://doi.org/10.1145/3736755

[16] FudanSELab. 2026. Train Ticket: A Benchmark Microservice System. https://github.com/FudanSELab/train-ticket. Accessed: 2026-02-26.

[17] Yazhuo Gao. 2025. An Framework for Fault Localization in Microservices. *Available at SSRN 5420754* (2025).

[18] Yufei Gao, Zhengong Cai, and Bowei Yang. 2026. Rcaflow: A workflow-informed hierarchical planning multi-agent system for root cause analysis. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 40. 300–308.

[19] Drishti Goel, Fiza Husain, Aditya Singh, Supriyo Ghosh, Anjaly Parayil, Chetan Bansal, Xuchao Zhang, and Saravan Rajmohan. 2024. X-lifecycle learning for cloud incident management using llms. In *Companion Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering*. 417–428.

[20] Google SRE Team. 2016. Effective Troubleshooting. https://sre.google/sre-book/effective-troubleshooting/. Accessed: 2024-12-01.

[21] Jiawei Gu, Xuhui Jiang, Zhichao Shi, Hexiang Tan, Xuehao Zhai, Chengjin Xu, Wei Li, Yinghan Shen, Shengjie Ma, Honghao Liu, Yuanzhuo Wang, and Jian Guo. 2024. A Survey on LLM-as-a-Judge. arXiv:2411.15594 [cs.CL]

[22] Xiaofeng Guo, Xin Peng, Hanzhang Wang, Wanxue Li, Huai Jiang, Dan Ding, Tao Xie, and Liangfei Su. 2020. Graph-based Trace Analysis for Microservice Architecture Understanding and Problem Diagnosis. In *Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*. ACM, New York, NY, 1387–1397.

[23] Priyanshu Gupta, Shashank Kirtania, Ananya Singha, Sumit Gulwani, Arjun Radhakrishna, Gustavo Soares, and Sherry Shi. 2024. MetaReflection: Learning Instructions for Language Agents using Past Reflections. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, 8369–8385.

[24] Yongqi Han, Qingfeng Du, Ying Huang, Jiaqi Wu, Fulong Tian, and Cheng He. 2024. The potential of one-shot failure root cause analysis: Collaboration of the large language model and small classifier. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*. 931–943.

[25] Adha Hrusto, Per Runeson, and Magnus C Ohlsson. 2024. Autonomous monitors for detecting failures early and reporting interpretable alerts in cloud operations. In *Proceedings of the 46th International Conference on Software Engineering: Software Engineering in Practice*. 47–57.

[26] Haiyu Huang, Jiewei Lyu, Zhihan Jiang, Jinyang Liu, Xiao He, Tieying Zhang, Wu Xiang, and Michael R Lyu. 2026. From General Agents to RCA Experts: A Self-Evolving Harness for Root Cause Analysis. *arXiv preprint arXiv:2608.25661* (2026).

[27] Taeyoon Kim, Woohyeok Park, Hoyeong Yun, and Kyungyong Lee. 2026. Why Do AI Agents Systematically Fail at Cloud Root Cause Analysis? arXiv:2602.09937 [cs.SE]

[28] Cheryl Lee, Tianyi Yang, Zhuangbin Chen, Yuxin Su, and Michael R. Lyu. 2023. Eadro: An End-to-End Troubleshooting Framework for Microservices on Multi-Source Data. In *Proceedings of the 45th IEEE/ACM International Conference on Software Engineering (ICSE)*. IEEE, 1750–1762.

[29] Haitao Li, Qian Dong, Junjie Chen, Huixue Su, Yujia Zhou, Qingyao Ai, Ziyi Ye, and Yiqun Liu. 2024. LLMs-as-Judges: A Comprehensive Survey on LLM-based Evaluation Methods. arXiv:2412.05579 [cs.CL]

[30] Mingjie Li, Zeyan Li, Kanglin Yin, Xiaohui Nie, Wenchi Zhang, Kaixin Sui, and Dan Pei. 2022. Causal Inference-Based Root Cause Analysis for Online Service Systems with Intervention Recognition. In *KDD '22: The 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining, Washington, DC, USA, August 14–18, 2022*, Aidong Zhang and Huzefa Rangwala (Eds.). ACM, 3230–3240. https://doi.org/10.1145/3534678.3539041

[31] Ruizhe Li, Mingxuan Du, Benfeng Xu, Chiwei Zhu, Xiaorui Wang, and Zhendong Mao. 2026. DeepResearch Bench II: Diagnosing Deep Research Agents via Rubrics from Expert Report. *arXiv preprint arXiv:2601.08536* (2026).

[32] Yichen Li, Yulun Wu, Jinyang Liu, Zhihan Jiang, Zhuangbin Chen, Guangba Yu, and Michael R. Lyu. 2025. COCA: Generative Root Cause Analysis for Distributed Systems with Code Knowledge. In *47th IEEE/ACM International Conference on Software Engineering, ICSE 2025, Ottawa, ON, Canada, April 26 - May 6, 2025*. IEEE, 1346–1358. https://doi.org/10.1109/ICSE55347.2025.00234 arXiv:2503.23051

[33] Zhijing Li, Jianbo Yu, Zhijun Huang, and Yusheng Huang. 2026. Surveying Root Cause Analysis Techniques: A Comprehensive Review of Aspects for Multi-Service Applications. *IEEE Transactions on Services Computing* 19, 1 (2026), 808–825. https://doi.org/10.1109/TSC.2025.3631913

[34] Shuai Liang, Pengfei Chen, Bozhe Tian, Gou Tan, Maohong Xu, Youjun Qu, Yahui Zhao, Yiduo Shang, and Chongkang Tan. 2026. MetaRCA: A Generalizable Root Cause Analysis Framework for Cloud-Native Systems Powered by Meta Causal Knowledge. In *Proceedings of the ACM International Conference on the Foundations of Software Engineering (FSE)*. https://doi.org/10.1145/3797069 arXiv:2603.02032 [cs.SE]

[35] Junpeng Lu, Baolei Pan, Jiashu Chen, Yun Feng, Jiacheng Hu, Yingxuan Peng, and Wei Chen. 2024. AgentLens: Visual Analysis for Agent Behaviors in LLM-Based Autonomous Systems. *IEEE Transactions on Visualization and Computer Graphics* (2024).

[36] Yu Luo, Rongchen Gao, Lu Teng, Xidao Wen, Jiamin Jiang, Qingliang Zhang, Yongqian Sun, Shenglin Zhang, Jiasong Feng, Tong Liu, et al. 2026. Graph of States: Solving Abductive Tasks with Large Language Models. *arXiv preprint arXiv:2603.21250* (2026).

[37] Yu Luo, Jiamin Jiang, Jingfei Feng, Lei Tao, Qingliang Zhang, Xidao Wen, Yongqian Sun, Shenglin Zhang, and Dan Pei. 2025. OpsAgent: An Evolving Multi-agent System for Incident Management in Microservices. *arXiv preprint arXiv:2510.24145* (2025).

[38] NVIDIA AI Blueprints. 2025. AIQ Research Assistant. https://github.com/NVIDIA-AI-Blueprints/aiq/tree/drb1.

[39] Changhua Pei, Zexin Wang, Fengrui Liu, Zeyan Li, Yang Liu, Xiao He, Rong Kang, Tieying Zhang, Jianjun Chen, Jianhui Li, et al. 2025. Flow-of-action: Sop enhanced llm-based multi-agent system for root cause analysis. In *Companion Proceedings of the ACM on Web Conference 2025*. 422–431.

[40] Luan Pham, Huong Ha, Xiuzhen Zhang, and Hongyu Zhang. 2026. TORAI: Multi-source Root Cause Analysis for Blind Spots in Microservice Service Call Graph. In *Proceedings of the ACM International Conference on the Foundations of Software Engineering (FSE)*. arXiv:2604.13522 [cs.SE]

[41] Bo Qiao, Liqun Li, Xu Zhang, Shilin He, Yu Kang, Chaoyun Zhang, Fangkai Yang, Hang Dong, Jue Zhang, Lu Wang, et al. 2023. Taskweaver: A code-first agent framework. *arXiv preprint arXiv:2311.17541* (2023).

[42] Evelien Riddell, James Riddell, Gengyi Sun, Michał Antkiewicz, and Krzysztof Czarnecki. 2026. Stalled, Biased, and Confused: Uncovering Reasoning Failures in LLMs for Cloud-Based Root Cause Analysis. In *Proceedings of the 3rd ACM International Conference on AI Foundation Models and Software Engineering (FORGE)*. arXiv:2601.22208 [cs.SE]

[43] Devjeet Roy, Xuchao Zhang, Rashi Bhave, Chetan Bansal, Pedro Las-Casas, Rodrigo Fonseca, and Saravan Rajmohan. 2024. Exploring llm-based agents for root cause analysis. In *Companion proceedings of the 32nd ACM international conference on the foundations of software engineering*. 208–219.

[44] Huasong Shan, Yuan Chen, Haifeng Liu, Yunpeng Zhang, Xiao Xiao, Xiaofeng He, Min Li, and Wei Ding. 2019. ϵ-Diagnosis: Unsupervised and Real-time Diagnosis of Small-window Long-tail Latency in Large-scale Microservice Platforms. In *The World Wide Web Conference, WWW 2019, San Francisco, CA, USA, May 13–17, 2019*, Ling Liu, Ryen W. White, Amin Mantrach, Fabrizio Silvestri, Julian J. McAuley, Ricardo Baeza-Yates, and Leila Zia (Eds.). ACM, 3215–3222. https://doi.org/10.1145/3308558.3313653

[45] Binpeng Shi, Yu Luo, Jingya Wang, Yongxin Zhao, Shenglin Zhang, Bowen Hao, Chenyu Zhao, Yongqian Sun, Zhi Zhang, Ronghua Sun, et al. 2025. Flowxpert: Expertizing troubleshooting workflow orchestration with knowledge base and multi-agent coevolution. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V. 2*. 4839–4850.

[46] Yifan Song, Da Yin, Xiang Yue, Jie Huang, Sujian Li, and Bill Yuchen Lin. 2024. Trial and error: Exploration-based trajectory optimization of LLM agents. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 7584–7600.

[47] Yongqian Sun, Rongchen Gao, Yu Luo, Wenwei Gu, Shenglin Zhang, Qingyi Guo, Qiuai Fu, Yaoliang Wu, and Dan Pei. 2026. OpsMem: Dual-Memory Reasoning with Cross-Memory Resonance for Failure Diagnosis. *arXiv preprint arXiv:2607.11357* (2026).

[48] Yongqian Sun, Yu Luo, Xidao Wen, Yuan Yuan, Xiaohui Nie, Shenglin Zhang, Tong Liu, and Xi Luo. 2025. TrioXpert: An Automated Incident Management Framework for Microservice System. In *40th IEEE/ACM International Conference on Automated Software Engineering, ASE 2025, Seoul, Korea, Republic of, November 16-20, 2025*. IEEE, 3239–3250. https://doi.org/10.1109/ASE63991.2025.00267

[49] Yongqian Sun, Tinghua Zheng, Xidao Wen, Weihua Kuang, Heng Liu, Shenglin Zhang, Chao Shen, Bo Wu, and Dan Pei. 2025. A multimodal intelligent change assessment framework for microservice systems based on large language models. In *Proceedings of the 33rd ACM International Conference on the Foundations of Software Engineering*. 378–388.

[50] ThinkDepth.ai. 2025. Deep Research. https://github.com/thinkdepthai/Deep_Research.

[51] Yifang Tian, Yaming Liu, Zichun Chong, Zihang Huang, Yiran Li, and Hans-Arno Jacobsen. 2026. GALA: Graph-Augmented LLM Agents for Root Cause Analysis and Incident Response in Microservices. *arXiv preprint arXiv:2608.08968* (2026).

[52] V Venktesh, Mandeep Rathee, and Avishek Anand. 2025. Trust but verify! a survey on verification design for test-time scaling. *arXiv preprint arXiv:2508.16665* (2025).

[53] Junle Wang, Xingchuang Liao, and Wenjun Wu. 2026. STAR: A Stage-attributed Triage and Repair Framework for RCA Agents in Microservices. arXiv:2605.15581 [cs.SE]

[54] Zefan Wang, Zichuan Liu, Yingying Zhang, Aoxiao Zhong, Jihong Wang, Fengbin Yin, Lunting Fan, Lingfei Wu, and Qingsong Wen. 2024. RCAgent: Cloud Root Cause Analysis by Autonomous Agents with Tool-Augmented Large Language Models. In *Proceedings of the 33rd ACM International Conference on Information and Knowledge Management,*

CIKM 2024, Boise, ID, USA, October 21–25, 2024, Edoardo Serra and Francesca Spezzano (Eds.). ACM, 4966–4974. https://doi.org/10.1145/3627673.3680016 arXiv:2310.16340

[55] Li Wu, Johan Tordsson, Erik Elmroth, and Odej Kao. 2020. MicroRCA: Root Cause Localization of Performance Issues in Microservices. In *NOMS 2020 – 2020 IEEE/IFIP Network Operations and Management Symposium*. IEEE. https://doi.org/10.1109/NOMS47738.2020.9110353

[56] Zhe Xie, Zeyan Li, Xiao He, Shenglin Zhang, Longlong Xu, Yuzhuo Yang, Tieying Zhang, Jianjun Chen, Rui Shi, and Dan Pei. 2026. FoundRoot: Towards Foundation Model for Root Cause Analysis via Structured Deep Thinking. In *Proceedings of the IEEE/ACM 48th International Conference on Software Engineering* (Rio de Janeiro, Brazil) *(ICSE '26)*. ACM. https://doi.org/10.1145/3744916.3787814

[57] Junjielong Xu, Qinan Zhang, Zhiqing Zhong, Shilin He, Chaoyun Zhang, Qingwei Lin, Dan Pei, Pinjia He, Dongmei Zhang, and Qi Zhang. 2025. Openrca: Can large language models locate the root cause of software failures?. In *The thirteenth international conference on learning representations*.

[58] Guangba Yu, Pengfei Chen, Hongyang Chen, Zijie Guan, Zicheng Huang, Linxiao Jing, Tianjun Weng, Xinmeng Sun, and Xiaoyun Li. 2021. MicroRank: End-to-End Latency Issue Localization with Extended Spectrum Analysis in Microservice Environments. In *WWW '21: The Web Conference 2021, Virtual Event / Ljubljana, Slovenia, April 19-23, 2021*, Jure Leskovec, Marko Grobelnik, Marc Najork, Jie Tang, and Leila Zia (Eds.). ACM / IW3C2, 3087–3098. https://doi.org/10.1145/3442381.3449905

[59] Guangba Yu, Pengfei Chen, Yufeng Li, Hongyang Chen, Xiaoyun Li, and Zibin Zheng. 2023. Nezha: Interpretable Fine-Grained Root Cause Analysis for Microservices on Multi-Modal Observability Data. In *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*. 553–565.

[60] Dylan Zhang, Xuchao Zhang, Chetan Bansal, Pedro Las-Casas, Rodrigo Fonseca, and Saravan Rajmohan. 2024. LM-PACE: Confidence estimation by large language models for effective root causing of cloud incidents. In *Companion Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering*. 388–398.

[61] Lingzhe Zhang, Tong Jia, Kangjin Wang, Chiming Duan, Minghua He, Rongqian Wang, Xi Peng, Meiling Wang, Gong Zhang, Renhai Chen, et al. 2026. Towards In-Depth Root Cause Localization for Microservices with Multi-Agent Recursion-of-Thought. *IEEE Transactions on Dependable and Secure Computing* (2026).

[62] Lingzhe Zhang, Tong Jia, Yunpeng Zhai, Leyi Pan, Chiming Duan, Minghua He, Mengxi Jia, and Ying Li. 2026. Agentic Memory Enhanced Recursive Reasoning for Root Cause Localization in Microservices. In *Proceedings of the 48th IEEE/ACM International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP '26)* (Rio de Janeiro, Brazil). ACM. https://doi.org/10.1145/3786583.3786853 arXiv:2601.02732

[63] Lingzhe Zhang, Tong Jia, Yunpeng Zhai, Leyi Pan, Chiming Duan, Minghua He, Pei Xiao, and Ying Li. 2026. Hypothesize-then-verify: Speculative root cause analysis for microservices with pathwise parallelism. In *Proceedings of the IEEE/ACM 48th International Conference on Software Engineering*. 21–25.

[64] Lingzhe Zhang, Yunpeng Zhai, Tong Jia, Chiming Duan, Siyu Yu, Jinyang Gao, Bolin Ding, Zhonghai Wu, and Ying Li. 2026. ThinkFL: Self-Refining Failure Localization for Microservice Systems via Reinforcement Fine-Tuning. *ACM Transactions on Software Engineering and Methodology* (2026). https://doi.org/10.1145/3789262 arXiv:2504.18776 Published online January 19, 2026 (Just Accepted).

[65] Songhan Zhang, Aoyang Fang, Yifan Yang, Ruiyi Cheng, Xiaoying Tang, and Pinjia He. 2025. DynaCausal: Dynamic causality-aware root cause analysis for distributed microservices. *arXiv preprint arXiv:2510.22613* (2025).

[66] Shenglin Zhang, Sibo Xia, Wenzhao Fan, Binpeng Shi, Xiao Xiong, Zhenyu Zhong, Minghua Ma, Yongqian Sun, and Dan Pei. 2024. Failure Diagnosis in Microservice Systems: A Comprehensive Survey and Analysis. *ACM Transactions on Software Engineering and Methodology* (2024).

[67] Shaokun Zhang, Ming Yin, Jieyu Zhang, Jiale Liu, Zhiguang Han, Jingyang Zhang, Beibin Li, Chi Wang, Huazheng Wang, Yiran Chen, and Qingyun Wu. 2025. Which Agent Causes Task Failures and When? On Automated Failure Attribution of LLM Multi-Agent Systems. In *Proceedings of the 42nd International Conference on Machine Learning (ICML) (Proceedings of Machine Learning Research, Vol. 267)*. PMLR. Spotlight.

[68] Wei Zhang, Hongcheng Guo, Jian Yang, Zhoujin Tian, Yi Zhang, Yan Chaoran, Zhoujun Li, Tongliang Li, Xu Shi, Liangfan Zheng, et al. 2024. mABC: multi-Agent Blockchain-Inspired Collaboration for root cause analysis in micro-services architecture. In *Findings of the Association for Computational Linguistics: EMNLP 2024*. 4017–4033.

[69] Xuchao Zhang, Supriyo Ghosh, Chetan Bansal, Rujia Wang, Minghua Ma, Yu Kang, and Saravan Rajmohan. 2024. Automated root causing of cloud incidents using in-context learning with GPT-4. In *Companion Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering*. 266–277.

[70] Xiao Zhang, Qi Wang, Mingyi Li, Yuan Yuan, Mengbai Xiao, Fuzhen Zhuang, and Dongxiao Yu. 2025. TAMO: Fine-Grained Root Cause Analysis via Tool-Assisted LLM Agent with Multi-Modality Observation Data in Cloud-Native Systems. *IEEE Transactions on Services Computing* 18, 6 (2025), 4221–4233. https://doi.org/10.1109/TSC.2025.3629066 arXiv:2504.20462

[71] Lecheng Zheng, Zhengzhang Chen, Jingrui He, and Haifeng Chen. 2024. MULAN: Multi-Modal Causal Structure Learning and Root Cause Analysis for Microservice Systems. In *Proceedings of the ACM on Web Conference 2024 (WWW)*. 4107–4116.

[72] Xiang Zhou, Xin Peng, Tao Xie, Jun Sun, Chao Ji, Dewei Liu, Qilin Xiang, and Chuan He. 2019. Latent Error Prediction and Fault Localization for Microservice Applications by Learning from System Trace Logs. In *Proceedings of the 27th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*. ACM, 683–694. https://doi.org/10.1145/3338906.3338961

[73] Mingchen Zhuge, Changsheng Zhao, Dylan R. Ashley, Wenyi Wang, Dmitrii Khizbullin, Yunyang Xiong, Zechun Liu, Ernie Chang, Raghuraman Krishnamoorthi, Yuandong Tian, Yangyang Shi, Vikas Chandra, and Jürgen Schmidhuber. 2025. Agent-as-a-Judge: Evaluate Agents with Agents. In *Proceedings of the 42nd International Conference on Machine Learning (ICML) (Proceedings of Machine Learning Research, Vol. 267)*. PMLR, 80569–80611.